



Agentic Coding: KI-gestützte Softwareentwicklung



**Joel Hintz & Jan
Domanski**

28.09.2026

Capgemini 

Wir stellen uns vor



Jan Domanski

Associate Business Analyst



jan.domanski@capgemini.com

[LinkedIn - Jan Domanski](#)

Mein Projektumfeld



*Business Analyse & GenAI
Enablement im
öffentlichen Sektor*

Capgemini bedeutet für mich



individuelle Entwicklung &
internationale Teamarbeit



Wir stellen uns vor



Joel Hintz
Software Engineer

Mein Projektumfeld



*Fullstack Software Entwicklung
im öffentlichen Sektor*

Capgemini bedeutet für mich



Starke Zusammenarbeit &
kontinuierliches Lernen



joel.a.hintz@capgemini.com

[LinkedIn - Joel Hintz](#)





Unser gemeinsamer Tag

**Genügend Pausen
zwischen durch**

**Mittagspause
13–14 Uhr**

- 1 **Software Engineering: Heute vs. Morgen**
Unsere Motivation
- 2 **Capgemini**
Unternehmensvorstellung
- 3 **Grundprinzipien von Agentic Coding**
Die wichtigsten Konzepte
- 4 **Deep Dive in Claude Code Teil 1**
Anfangen mit Claude Code
- 5 **Hands-on Teil 1**
Einführung in Aufgabenstellung, die ersten zwei Aufgaben
- 6 **Deep Dive in Claude Code Teil 2**
Skills, Agents und Workflows
- 7 **Hands-on Teil 2**
Aufgabe 3
- 8 **Abschluss**
Berufsperspektive bei Capgemini und Feedback



Software Engineering: Heute vs. Morgen



Software Engineering - Heute

Code als Hauptartefakt, der Mensch als Autor.



CHARAKTERISTIKA

Code-Erstellung als primäre Tätigkeit

Tippen und Boilerplate Code dominieren den Alltag.

Tiefe Sprach- und Framework-Kenntnis

Wert entsteht über Spezialisierung in Stack X.

Sequenzieller Workflow

Design → Code → Test → Deploy, oft in Wochen-Zyklen.

Tests häufig nachgelagert

TDD eher Ideal als gelebte Praxis.



BEWERTUNG & KARRIERE

Produktivität ≈ Erfahrung

Output wird in Tickets und Features gemessen.

Senioritätswachstum durch Wiederholung

Mehr gesehene Bugs = bessere Intuition.

Karrierepfad „Stack-Spezialist“

Tiefe in einer Technologie als Differenzierung.

Wissen ≈ Verfügbarkeit

Wer es im Kopf hat, schreibt es schneller.



Software Engineering - Morgen

Die Kernaufgabe verschiebt sich vom Schreiben zum Verstehen, Spezifizieren und Verifizieren.

Vom Autor zum Architekten

KI generiert und refaktoriert Code. Der Mensch trifft Architekturentscheidungen.

Spec- & Test-driven Workflows

Tests werden zur primären Spezifikation - sie sagen der KI, was richtig ist.

Agents als Kollaborateure

Iterationen schrumpfen von Tagen auf Minuten. Mehrere Agents arbeiten parallel.

Review wird zur Kernkompetenz

Der Skill ist nicht „Code schreiben“, sondern „erkennen, ob ein Vorschlag im System richtig ist“.



Skills & Fokuspunkte

Worauf Softwareentwickler jetzt setzen sollten.



Systemverständnis

Architektur, Datenflüsse, Integration



Domänen-Expertise

Wenn Code günstig wird, ist „zu wissen, welcher Code richtig ist“ der Burggraben.



Präzise Spezifikation

Prompts ausformulieren: was, warum, unter welchen Constraints.



SWE-Grundlagen

Clean Code, Design Patterns - die Sprache, in der man mit Agents präzise spricht.



Code-Review & Verifikation

KI-Output kritisch prüfen.



Kritisches Denken & AI-Literacy

Wann KI vertrauen, wann nicht. Halluzinationen früh erkennen.



Unternehmens- vorstellung





Capgemini in Zahlen

Capgemini ist einer der weltweit führenden Anbieter von:

- Management- und IT-Beratung
- Technologie-Services
- Digitaler Transformation



39 Jahre

Durchschnittsalter



50

Länder



22,5 Mrd.

Euro Umsatz in 2025



120

Nationalitäten



420.000

Mitarbeitende



380

Offices weltweit

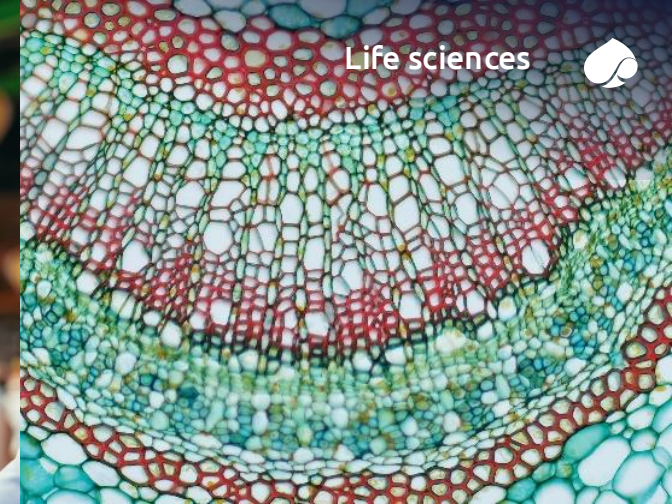
Unsere Branchen



Banking
and capital markets



Retail



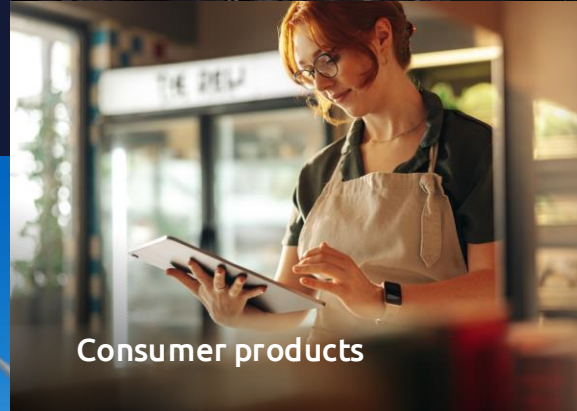
Life sciences



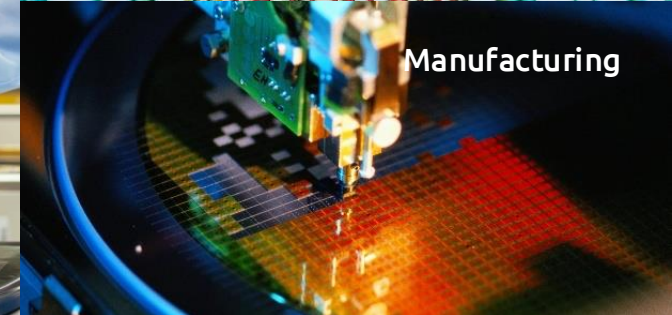
Aerospace
and defense



Energy & utilities



Consumer products



Manufacturing



Automotive



Healthcare



Insurance



Media and entertainment



Telecoms



High-tech



Public sector



Hospitality and travel



Grundprinzipien von Agentic Coding



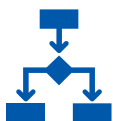
Analytische vs. Generative KI

Vom Erkennen von Mustern zur Erzeugung neuer Inhalte

Analytische KI



Strukturierte Daten
(z. B. Transaktionen, Tabellen, ...)



Zusammenhänge & Muster
in Daten erkennen

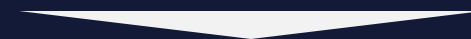


Konkrete Entscheidungen & Bewertungen
(z.B. Risiko-Score, Klassifikation, ...)

Generative KI



Prompt & Kontext
(z. B. lose Frage, Dokumente, ...)



Inhalte auf Basis von
Wahrscheinlichkeiten generieren



Fertige Inhalte für den Nutzer
(z. B. Text, Code, Bild, ...)



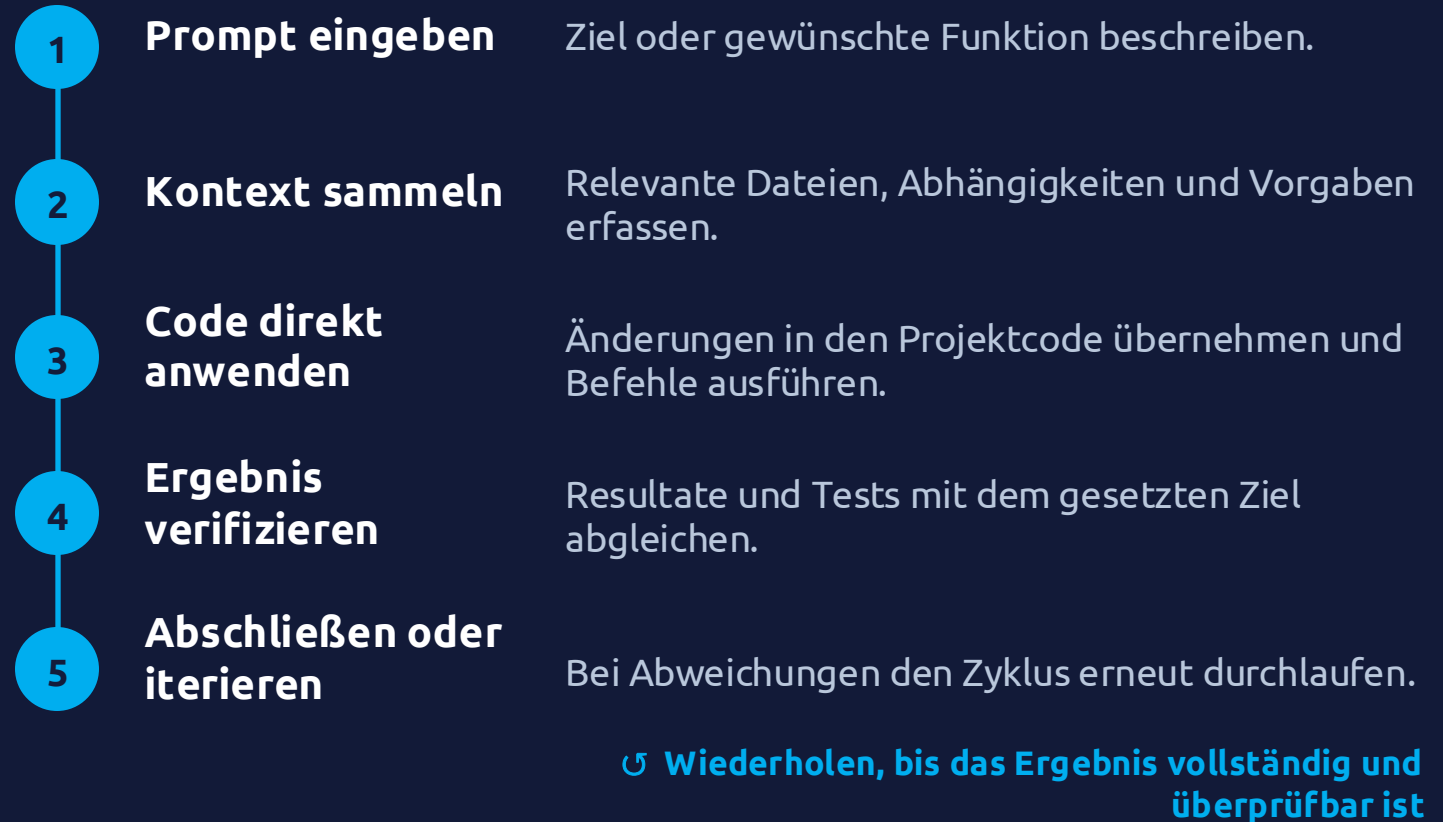
Agentic Coding: Definition und Funktionsweise

AI Coding Assistants sind direkt in die IDE integrierte LLM-Werkzeuge, die aus Prompts Codevorschläge erzeugen.

Agentic Coding entsteht, wenn das System nicht nur Code vorschlägt, sondern Kontext sammelt, Werkzeuge nutzt und Änderungen direkt im Projektcode vornimmt und überprüft.

Der Mensch definiert Ziel, Rahmen und Freigabe.

Der agentische Arbeitszyklus





Einsatzmöglichkeiten von Agentic Coding

TYPISCHE AUFGABEN IM ENTWICKLUNGSALLTAG

— Entwickeln & verändern

Code generieren, Funktionen umsetzen und Code migrieren

— Fehler beheben

Debugging und direkte Korrekturen im Projektcode

— Verstehen & verbessern

Code erklären, Systeme analysieren und Code formatieren

— Qualität sichern

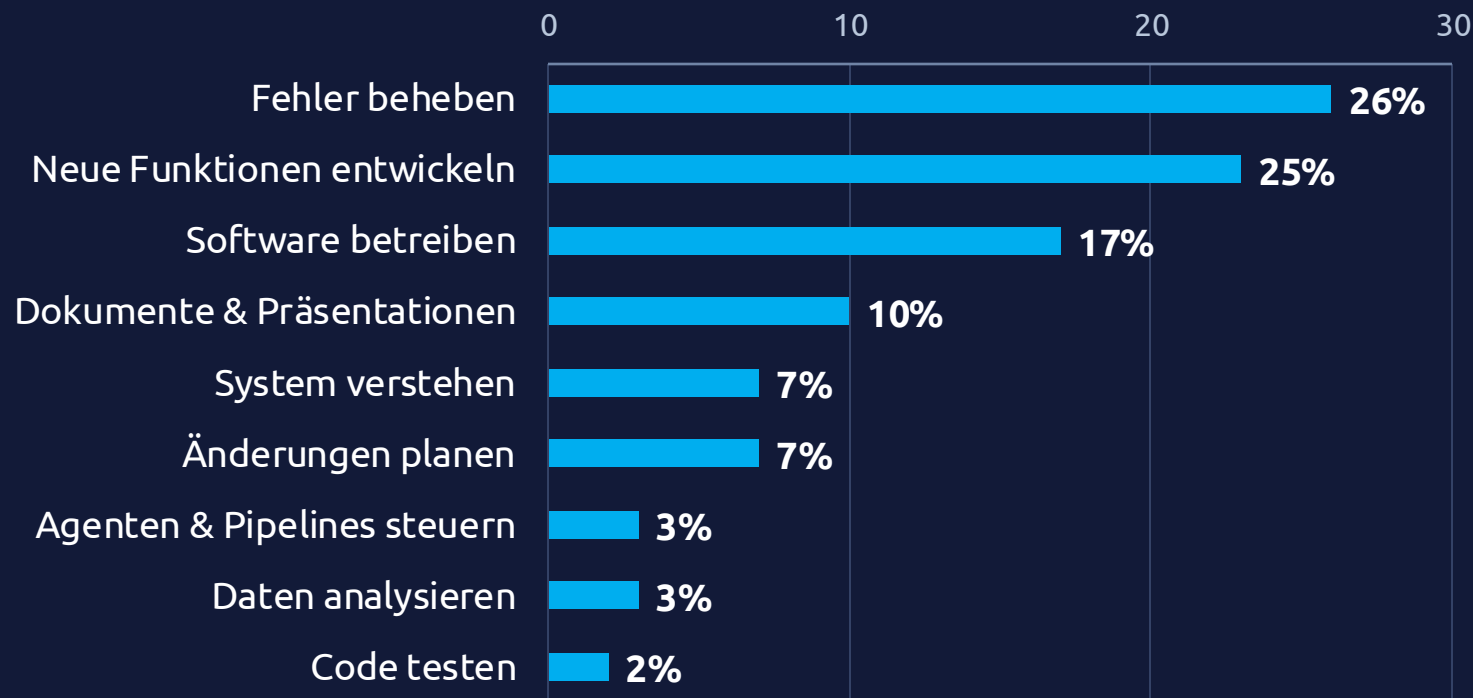
Testfälle erzeugen, Tests ausführen und Ergebnisse prüfen

— Dokumentieren

Kommentare und technische Dokumentation erstellen

Wie Coding Agents heute eingesetzt werden

Anteil beobachteter Sitzungen



Quelle: Anthropic – How Claude Code is used in practice



Tokens: Bausteine, mit denen ein LLM Sprache verarbeitet

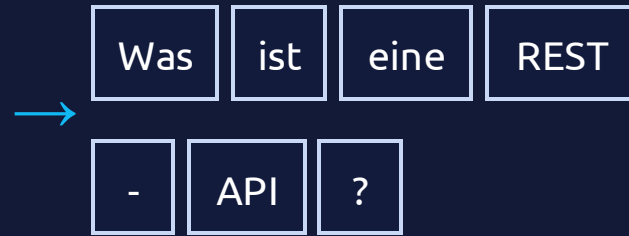
Ein Token ist ein Wort, Wortteil oder Zeichen. Das Modell sieht nur Tokens.

1 PROMPT

„Was ist eine REST-API?“

Text als Eingabe

2 TOKENIZER



zerlegt Text in verarbeitbare Einheiten

3 LLM

sagt das nächste Token voraus

Token für Token

4 AUSGABE

„Eine REST-API ist ...“

lesbarer Text

WARUM DAS WICHTIG IST

Bezahlt werden Tokens, nicht Fragen.

Deutsch braucht oft mehr Tokens, da Zusammengesetzte Wörter in viele Tokens zerlegt werden · Code enthält viele Zusatz-Tokens



System Prompt setzt den Rahmen – User Prompt gibt die Aufgabe

System Prompt

- **Rolle & Verhalten** festlegen
- Gilt für den **gesamten Dialog**

Du bist ein Unternehmensberater, der für seine analytischen Fähigkeiten bekannt ist, um Verbesserungsmöglichkeiten in einem Unternehmen zu erkennen. Du hast einen scharfen Blick für Details und ein Talent dafür, Ineffizienzen oder Engpässe aufzuspüren, die Wachstum und Erfolg behindern könnten. ...

AI System



User Prompt

- **Konkrete Aufgabe** stellen
- Steuert **Inhalt & Richtung**

Analysiere die aktuelle Marktposition eines mittelständischen Unternehmens in der Energiebranche.

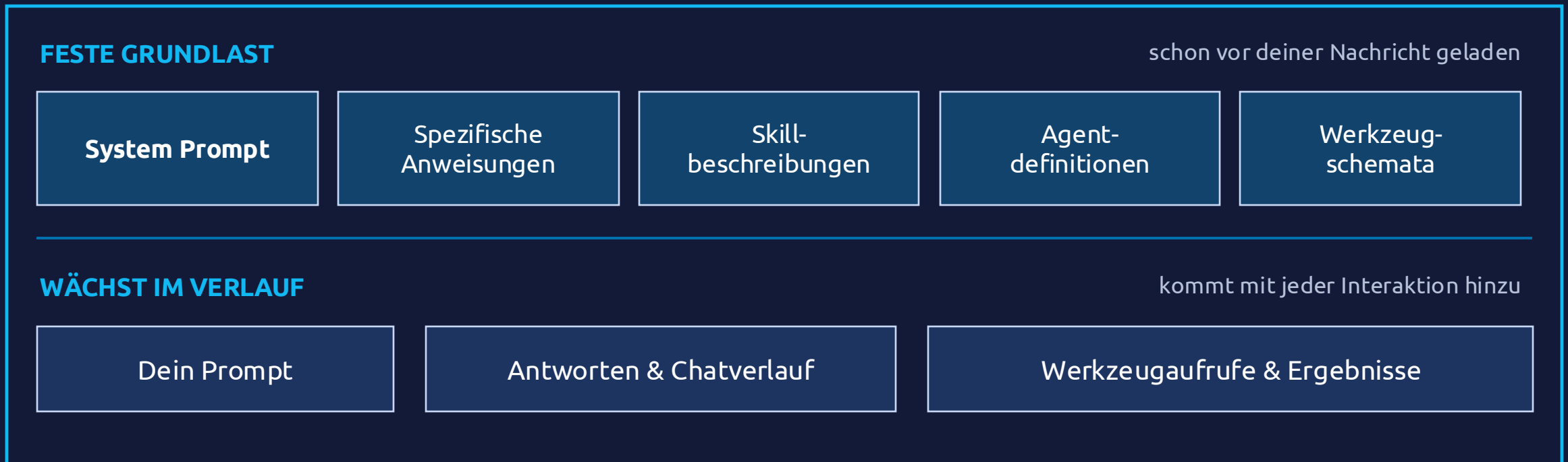
Schlage Maßnahmen zur Effizienzsteigerung in der internen Logistik eines Unternehmens (200 Mitarbeitende) vor.



Das Kontextfenster ist der Arbeitsbereich des Modells

Vor dem ersten Prompt ist ein Teil des Kontextfensters bereits belegt

KONTEXTFENSTER = BEGRENZTE ANZAHL AN TOKENS, DIE BEI EINER ANFRAGE GLEICHZEITIG VERARBEITET WERDEN



MERKSATZ **Feste Grundlast und Gesprächskontext werden bei jeder Anfrage gemeinsam verarbeitet.**



Konversation Verwalten - Turns

WAS IST EIN TURN?

Ein Turn ist ein vollständiger Anfrage-Antwort-Schritt. Bei jeder neuen Frage wird der bisherige Verlauf erneut gesendet.

TURN 1 **Setup** + **Frage 1**

TURN 2 **Setup** + Frage 1 + Antwort 1 + **Frage 2**

TURN 3 **Setup** + Frage 1 + Antwort 1 + Frage 2 + Antwort 2 + **Frage 3**

MERKSATZ Jeder neue Turn enthält den bisherigen Verlauf – plus die neue Frage.



Konversation Verwalten - Cross Task Falle

Einer der häufigsten und am leichtesten nachvollziehbaren Fehler bei der Nutzung von LLMs.

Wird Aufgabe 3 in derselben Unterhaltung wie Aufgabe 1 und 2 bearbeitet, schleppt sie deren gesamten Verlauf bei jedem Turn mit.

EIN CHAT Aufgabe 1 ✓ + Aufgabe 2 ✓ + **Aufgabe 3 aktiv**

Abgeschlossener Kontext wird weiterhin übertragen – ohne Mehrwert für Aufgabe 3.

MAN ZAHLT DOPPELT

01 KOSTEN

Historische Tokens werden erneut gesendet.
Mehr Verbrauch · höhere Kosten

02 QUALITÄT

Themen konkurrieren um Aufmerksamkeit.
Fokusverlust · Vermischung · Context Rot

DIE LÖSUNG Für jede neue, unabhängige Aufgabe eine neue Unterhaltung starten.

FAUSTREGEL Neue Aufgabe = neue Unterhaltung.



Konversation Verwalten - Context Rot

Mehr Kontext macht das Modell nicht intelligenter, sondern dümmer.

AUFMERKSAMKEIT IST EIN BEGRENZTES BUDGET

- 01 Relevante Inhalte werden von Ballast überlagert.
- 02 „Needle in a Haystack“: Wichtiges ist schwerer wiederzufinden.
- 03 Die Genauigkeit sinkt oft schon vor dem Kontextlimit.

3

SEITEN

relevant → erfassbar

300

SEITEN

viel → überfliegen

WOHER ES KOMMT Lange Chats · mehrere Aufgaben · große Dateien · Planung + Umsetzung · überladene Konfigurationen

KONTEXT ZURÜCKSETZEN

/compact

Gleiches Thema, Chat zu lang

Verdichtet den Verlauf und erhält Wichtiges.

/clear · neuer Chat

Neues oder anderes Thema

Entfernt den alten Kontext vollständig.

FAUSTREGEL Thema bleibt → verdichten. Thema wechselt → zurücksetzen.



Konversation Verwalten – Do's and Don'ts

✓ DO

1. Neuer Chat für jedes neue Thema

Nutze für die neue Aufgabe den kleinstmöglichen Kontext.

2. Lange Unterhaltungen zum selben Thema komprimieren

Fasse den bisherigen Verlauf zusammen, bevor er zu umfangreich wird.

3. Umfangreiche Recherche aus dem Hauptchat heraushalten

Lagere intensive Recherche an einen Sub-Agenten aus.

4. Aufgabe abschließen und anschließend neu starten

Nicht mehr benötigter Kontext sollte nicht mitgeführt werden.

✗ DON'T

1. Einen Chat mit 50 Turns für fünf verschiedene Themen nutzen

Frühere Themen beeinträchtigen die nächste Aufgabe.

2. Einen Chat bis an das Kontextlimit füllen

Das Modell wird schon vor Erreichen des Kontextlimits weniger leistungsfähig.

3. Ganze Dateien vorsichtshalber einfügen

Die Inhalte werden bei jedem Turn erneut verarbeitet, auch wenn sie nicht benötigt werden.

4. Den Chat als dauerhaftes Gedächtnis behandeln

Das Modell ist zustandslos und liest den gesamten Kontext immer wieder neu.



Prompting Techniken

01 ZERO-SHOT

Keine Beispiele

Nur die Aufgabe – ohne Beispiele.

TYPISCHER EINSATZ

Standardfragen ·
Zusammenfassen · Übersetzen

Stärke Schnell & einfach

02 FEW-SHOT

Wenige Beispiele

Beispiele zeigen Muster und Zielformat.

TYPISCHER EINSATZ

Klassifikation · Datenextraktion
· Formatierte Ausgaben

Stärke Präziser steuerbar

03 CHAIN-OF-THOUGHT

Schrittweise lösen

Zwischenschritte strukturieren, dann das Ergebnis formulieren.

TYPISCHER EINSATZ

Logik & Mathematik ·
Problemlösung ·
Entscheidungen

Stärke Stark bei komplexen Fragen

04 PROMPT CHAINING

Prompts verknüpfen

Aufgabe in Schritte teilen;
Output wird zum nächsten Input.

TYPISCHER EINSATZ

Agentensysteme · Content-Erstellung ·
Prozessautomatisierung

Stärke Höchste Kontrolle



Was ist ein Skill?

DEFINITION

Ein Skill beschreibt, **wie und wann** eine bestimmte Aufgabe ausgeführt werden soll.

So führen KI-Agenten klar definierte Aufgaben **konsistent, effizient und wiederholbar** aus.

WIEDERVERWENDBAR GEBÜNDELT

- 01 Fachwissen und Regeln**
- 02 Vorlagen und Referenzen**
- 03 Prozessschritte**
- 04** optional unterstützende Skripte oder Tools

PROGRESSIVE DISCLOSURE

Der Agent lädt Wissen stufenweise – nur so tief wie nötig.

- 01 Metadaten** Name und Einsatzzweck · immer bekannt
- 02 SKILL.md** vollständiger Inhalt · erst bei Bedarf
- 03 Referenzen & Tools** separat · gezielt nachgeladen

VORTEIL Kompakter Startkontext · volles Wissen bei Bedarf.

WANN IST EIN SKILL DIE RICHTIGE LÖSUNG?

- wiederkehrende Anweisungen
- Vorlagen, Referenzen oder Skripte
- umfangreiches Verfahrenswissen nur bei Bedarf
- klar abgegrenzter, mehrstufiger Workflow



Was ist ein Agent?

DEFINITION

Ein Agent ist ein **KI-Arbeiter** mit einer klaren Rolle, einem Ziel und definierten Fähigkeiten. Er bearbeitet Aufgaben eigenständig und **entscheidet, welche Fähigkeit wann zum Einsatz kommt**.

SO ARBEITET EIN AGENT

- | | | |
|----|-------------------------|--|
| 01 | ROLLE & ZIEL | Definierte Rolle und Zielsetzung – z. B. Reviewer, Architect, Developer. |
| 02 | KONTEXT | Hat eigenen Arbeitskontext |
| 03 | ZUGRIFF | Greift auf Skills, Tools und Guardrails zu. |
| 04 | ORCHESTRIERUNG | Wählt und kombiniert die passende Fähigkeit für jeden Arbeitsschritt. |

WISSEN STECKT NICHT IM AGENTEN. Es lebt in den Skills, die er nutzt.

BEISPIEL

Frontend-Agent

ROLLE Frontend Developer

VERFÜGBARE SKILLS

- 01 **Angular-Komponenten-Konventionen**
- 02 **Playwright-Tests**
- 03 **UI-Accessibility-Checks**



Viele Coding-Agenten - ein übertragbares Fundament

UNTERSCHIEDLICHE SCHWERPUNKTE – GEMEINSAME ARBEITSPRINZIPIEN

LÖSUNG

BEISPIELHAFTER SCHWERPUNKT

GitHub Copilot	Integration in GitHub-, Issue- und Pull-Request-Workflows
Cursor	Agentisches Arbeiten direkt im AI-nativen Editor
Claude Code	Terminalbasierte Arbeit und Automatisierung im Repository
OpenAI Codex	Aufgaben lokal oder an Cloud-Agenten delegieren
Windsurf Cascade	Integrierter Editor-Workflow mit Planung, Rules und Memories
Gemini Code Assist	Entwicklungsunterstützung im Google-Cloud-Umfeld

WAS IHR BEI JEDEM TOOL WIEDERVERWENDEN KÖNNT

Aufträge präzisieren: Ziel und Umfang formulieren.

Kontext bereitstellen: Relevanten Code, Projektwissen und Regeln zugänglich machen.

Arbeit steuern: Planung einfordern, Aufgaben zerlegen und Berechtigungen begrenzen.

Skills & Agents: Anbieterübergreifende Konzepte – Umsetzung und Verfügbarkeit variieren.

Beim Toolwechsel lernt ihr eine neue Bedienung – nicht die Grundlagen neu.



Deep Dive in Claude Code 1



Modell und Effort-Level gezielt kombinieren

1. Passendes Modell auswählen

MODELL → FÄHIGKEITEN

SCHNELL – Haiku 4.5

Einfache Fragen · Kurzfassungen · kleine Änderungen
Schnell & ressourcenschonend

AUSGEWOGEN – Sonnet 5

Analysen · Programmierung · Refactoring
Balance aus Tempo, Qualität & Aufwand

LEISTUNGSSTARK – Opus 5 & Fable 5.1

Komplexe Probleme · Architektur · agentische Workflows
Maximale Fähigkeiten für anspruchsvolle Aufgaben

+
unabhängig
wählbar

2. Bearbeitungstiefe steuern

EFFORT → INTENSITÄT

schneller ————— mehr Tiefe



Effort steuert die Bearbeitungstiefe – nicht die Token-Grenze.

Das Modell bestimmt die Fähigkeiten. Das Effort-Level bestimmt, wie intensiv sie für die Aufgabe eingesetzt werden.



CLAUDE.md: Onboarding für jede Session

Ohne CLAUDE.md beginnt Claude Code in jeder Session wieder bei null.

OHNE CLAUDE.md

Claude muss Codebasis, Abhängigkeiten und bereits umgesetzte Features erneut erkunden. Fehlender Kontext begünstigt Annahmen und erschwert die Steuerung.

WAS CLAUDE.md ÄNDERT

Die Markdown-Datei liegt im Projektstamm und wird zu Beginn jeder Session automatisch gelesen. Ihr Inhalt wird dem Prompt hinzugefügt – wie ein dauerhaftes Onboarding-Skript für die Codebasis.

DREI EBENEN FÜR ANWEISUNGEN

- | | | |
|----|------------------|--|
| 01 | USER | Persönliche Präferenzen, die auf deinem Rechner projektübergreifend gelten. |
| 02 | PROJECT | Gemeinsame Projektdatei, die mit dem Team geteilt und ins Repository eingecheckt wird. |
| 03 | SUBMODULE | Lokale Anweisungen nur für einen Teilbereich – zum Beispiel das Frontend. |

CLAUDE.md macht Projektwissen explizit, wiederholbar und im Team steuerbar.



Rules: Kontext gezielt steuern

Mehrere spezialisierte Rule-Dateien liefern nur den Kontext, der gerade gebraucht wird.

NACH THEMA TRENNEN

Eine Datei pro Thema hält Regeln klar und domänenspezifisch.

NUR BEI BEDARF LADEN

Path-spezifische Regeln im Frontmatter laden passende Anweisungen gezielt in den Kontext.

GÜLTIGKEITSBEREICH

Guidelines gelten nur für die Dateien, auf deren Pfade die jeweilige Rule verweist.

BEISPIELE

- testing
- api-design
- code-style
- security

Rules reduzieren Kontextballast und machen Anweisungen präzise dort verfügbar, wo sie gelten.



Worauf hat Claude Code Zugriff?

Wenn du Claude in einem Verzeichnis startest, erhält Claude Code Zugriff auf:

DEIN PROJEKT

Dateien im Verzeichnis und in Unterverzeichnissen – sowie mit deiner Erlaubnis weitere Dateien.

DEIN TERMINAL

Alle Befehle, die du selbst ausführen könntest: Build-Tools, Git, Paketmanager, Systemwerkzeuge und Skripte.

DEIN GIT-STATUS

Aktueller Branch, uncommittete Änderungen und die jüngste Commit-Historie.

DEINE CLAUDE.md

Projektspezifische Anweisungen, Konventionen und Kontext, die Claude in jeder Session kennen soll.

AUTO MEMORY

Automatisch gespeicherte Projektmuster und Präferenzen.

KONFIGURIERTE EXTENSIONS

MCP-Server, Skills, Subagents und Workflows

Claude Code arbeitet mit deinem Projektkontext – und nur innerhalb der Berechtigungen, die du gewährst.

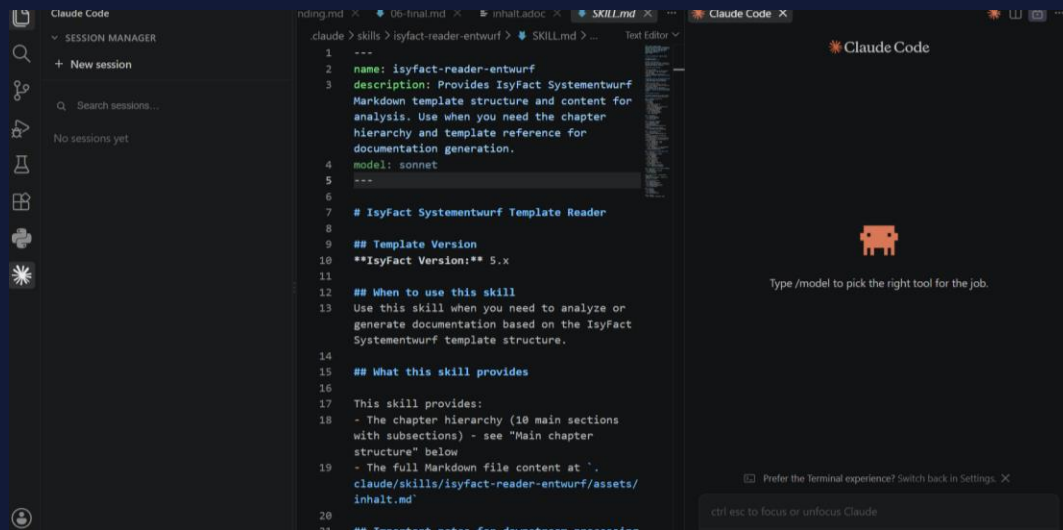


IDE oder CLI

Beide nutzen Claude Code – nur über unterschiedliche Interfaces

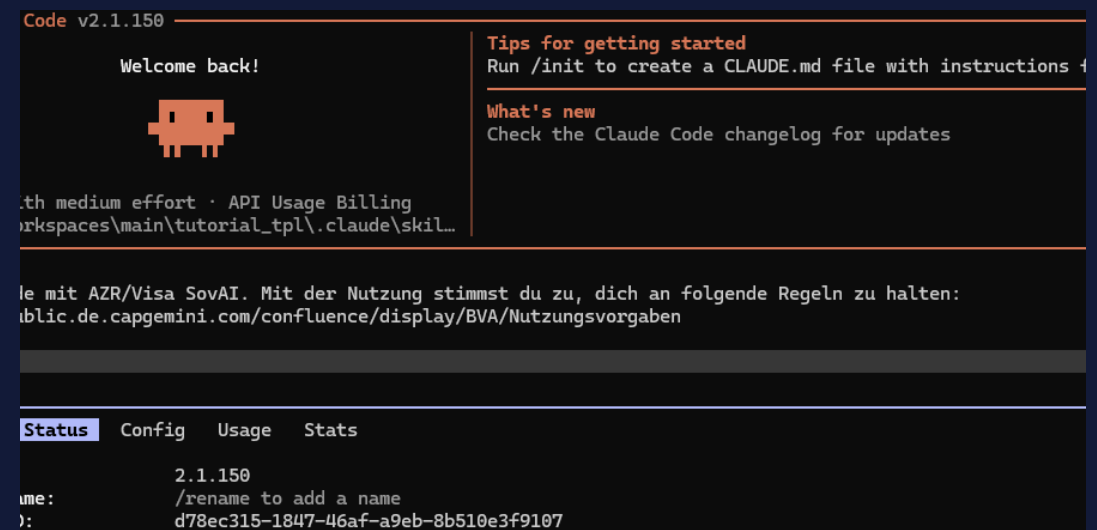
IDE-INTEGRATION

Für eine vertraute visuelle Experience direkt im Editor



CLI

Die ursprüngliche „native“ Oberfläche von Claude Code



Einige Funktionen sind in der CLI direkter oder ausschließlich dort verfügbar



Claude Code: wichtigste Befehle im Überblick

KONTEXT & GESPRÄCH

/clear — Gespräch leeren und neu starten.

/compact [Hinweis] — Verlauf mit Fokusvorgabe verdichten.

/context — Kontextbelegung und Spartipps anzeigen.

/fork [Name] — Gespräch ab hier verzweigen.

/rewind — Gespräch oder Code zurücksetzen (/checkpoint).

/export [Dateiname] — Gespräch als Text exportieren.

/copy — Antwort oder Codeblock kopieren.

MODELL & LEISTUNG

/model [Modell] — Modell in der Session wechseln.

/effort [Stufe] — Denkaufwand wählen.

SESSION & NUTZUNG

/cost — Token und API-Kosten anzeigen.

/usage — Limits und Rate-Limit prüfen.

/stats — Nutzungsverlauf auswerten.

SPEICHER & PROJEKT

/init — CLAUDE.md fürs Projekt erstellen.

/memory — Dauerhafte Erinnerungen verwalten.

/hooks — Automationen an Tool-Ereignisse hängen.

/permissions — Tool-Rechte verwalten (/allowed-tools).

/config — Theme, Modell und Ausgabe einstellen (/settings).

MCP & INTEGRATIONEN

/mcp — Server, Verbindungen und OAuth verwalten.

/plugin — Plugins installieren, entfernen oder suchen.

/reload-plugins — Plugins ohne Neustart neu laden.

AGENTEN & AUFGABEN

/agents — Subagenten und Teams verwalten.

/tasks — Hintergrundaufgaben steuern.

/plan [Beschreibung] — Planmodus mit Aufgabe starten.

/btw [Frage] — Nebenfrage ohne Kontextzuwachs stellen.

/skills — Verfügbare Skills auflisten.

KONTO & SYSTEM

/login — Bei Anthropic anmelden.

/logout — Abmelden.

/doctor — Installation prüfen; Fehler finden.

/status — Version, Modell, Konto und Verbindung.



Hands-on Teil 1



Organisation

Gemischte Teams bearbeiten drei aufeinander aufbauende Aufgaben.

TEAM-SETUP

4–5 Personen pro Team

MINDESTENS 1 Informatiker pro Team

NICHT NUR Informatiker im Team

AUFGABEN

1 Erkunden, dokumentieren, Bug beheben

2 Freie Termin-Slots implementieren

THEORIE-TEIL → Voraussetzung für Aufgabe 3

3 **Agentic-Coding-Workflow bauen**
und das Feature nachvollziehbar umsetzen

OPTIONAL Jede Aufgabe enthält einen optionalen Teil



Setup für die Übung

Repository und Claude-Einstellungen

REPOSITORY

[JoelHintz/appointment-api](#)

Aufgabenstellungen und Code auf GitHub

API-KEY

c2stYW50LWFwaTAzLWp1SFdRUVQ4RUNYdW5peW5Je
ElsaVY1ZzktClY4VzZEWnJPejZKMDFQWFZSa1hKLQpIYV
FzTExUeDhuOWRhUFRQNlInSEt6UGhXR3diREVWNmF2
M3hDMUEtCkhxUGZUQUFB4oCL

MODELL

[/model](#)

Sonnet auswählen

EFFORT

[/effort](#)

Sicherstellen, dass high eingestellt ist



Das Projekt: Appointment API

App-Struktur

appointment-api/

01 **.claude/**

agents/
skills/

02 **src/**

appointments/
offices/

03 **tasks/**

Workshop-Aufgaben

04 **tickets/**

Ticket für Aufgabe 3

ZWECK

Agenten & Skills für spätere
Aufgaben

Application Code
TypeScript · NestJS

Aufgaben für den praktischen
Teil

Umsetzung der dritten
Aufgabe

Domänenmodell



Vier Regeln bestimmen jeden Termin

**Start zur
vollen
Stunde**

09:00, 10:00, ...

**Dauer
1 Stunde**

feste Slotlänge

**Genau ein
Office**

office-spezifisch

**Keine
Doppel-
belegung**

gleiches Office +
Zeit



Aufgabe 1: Erkunden, dokumentieren, Bug beheben

EUER KONKRETER AUFTRAG

Erschließt die Appointment API, haltet das Projektwissen in einer CLAUDE.md fest und behebt einen fachlich ungültigen Eingabefall.

Ergebnis: eine kompakte CLAUDE.md und Bugfix

1 Anwendung verstehen

- Verhalten zuerst in Swagger erkunden
- Request-Weg mit Claude nachvollziehen
- DTO, Service und Controller abgrenzen

2 CLAUDE.md erstellen

- Sechs vorgegebene Abschnitte ausfüllen
- Selbst schreiben, danach durch Claude prüfen
- Kompakt halten: eine Bildschirmseite genügt

3 Bug finden & beheben

- Fachlich ungültige Eingaben in Swagger sammeln
- Ursache vor der Änderung in einem Satz benennen
- Fix an der richtigen Stelle umsetzen

ABNAHME

CLAUDE.md vollständig • ein Bug in Swagger nachgeprüft • Ursache und Ort der Fachlogik erklärbar

Danach gemeinsam den git diff lesen und jede Änderung begründen können

OPTIONAL

Test • frische Session • zweiter Fix



Aufgabe 2: Freie Termin-Slots mit Prompting implementieren

EUER KONKRETER AUFTRAG

Implementiert (promptet) einen Endpunkt, der für ein Amt und einen Tag alle freien Ein-Stunden-Slots zurückgibt.

GET /offices/id/availability?date=2027-06-30

Ergebnis: freie startHour-Werte

1 Was ihr dafür umsetzt

- REST-Endpoint im vorhandenen Projektstil
- Query-DTO für date
- Response-DTO für die freien Slots
- @ApiProperty() an beiden DTOs zum testen im Swagger

2 Fachlogik

- Slots buchbar: opensAtHour bis closesAtHour
- date und startHour bleiben Ortszeit des Amts
- Bereits gebuchte Slots werden ausgeschlossen

3 Testen in Swagger

- 8–16 Uhr frei → 8 bis 15
- 10 Uhr gebucht → 10 fehlt
- Ungültiges Datum → 400
- Unbekanntes Amt → 404

VORGEHEN Aufgabe lesen → offene Details entscheiden → umsetzen → Swagger prüfen

Danach gemeinsam den git diff lesen und erklären können, wo die Fachlogik liegt.

OPTIONAL

Unit-Tests • Stornieren • vergangene Daten



Deep Dive in Claude Code 2



Skills

WIEDERVERWENDBARE ANLEITUNGEN

Skills bündeln Fachwissen, Checklisten und wiederkehrende Abläufe. Claude lädt einen passenden Skill automatisch, sobald er für die Aufgabe relevant ist.

DIREKTER AUFRUF

Alternativ lässt sich ein Skill gezielt mit `/skill-name` starten.

ABLAGEN & STRUKTUR

Projektweite Skills liegen unter `.claude/skills/<name>/SKILL.md`.

Prompte dir den ersten Entwurf des Skills:

Erstelle einen projektweiten Skill „api-review“ unter `.claude/skills/api-review/`. Lege eine SKILL.md mit Name und Beschreibung an. Der Skill soll REST-Endpunkte auf Benennung, Eingabevalidierung, Fehlerformate und Tests prüfen. Claude soll ihn automatisch bei API-Reviews einsetzen; zusätzlich soll er mit `/api-review` aufrufbar sein. Gib Befunde priorisiert mit konkreten Verbesserungsvorschlägen aus.

Beispiel für eine Skill-Datei:

```
---
name: api-review
description: Prüft REST-APIs auf Konventionen, Sicherheit und Testbarkeit
---

Prüfe Benennung, Eingabevalidierung, Fehlerformate und Tests.
Gib priorisierte, konkrete Verbesserungsvorschläge aus.
```



Subagents

AUTOMATISCHE DELEGATION

Passt eine Aufgabe zur Beschreibung eines Subagents, delegiert Claude sie automatisch. Der Subagent arbeitet selbstständig und liefert das Ergebnis zurück.

DIREKTER AUFRUF

Alternativ lässt sich ein Subagent gezielt mit `@AgentName` starten.

SITZUNG & ABLAGE

Subagents arbeiten innerhalb einer einzelnen Sitzung. Eigene Agent-Dateien liegen in Claude Code unter `.claude/agents`.

Prompte dir den ersten Entwurf des Agenten:

Erstelle einen persönlichen Subagenten „code-reviewer“ unter `~/claude/agents/`. Er soll Code auf Qualität, Sicherheit, Wartbarkeit und bewährte Vorgehensweisen prüfen. Erkläre jeden Befund, nenne die betroffene Stelle und gib eine konkrete Verbesserungsempfehlung. Der Agent darf Dateien nur lesen und soll Sonnet verwenden.

Beispiel für eine Agentdatei:

```
---
name: code-reviewer
description: Prüft Code auf Qualität, Sicherheit und bewährte Vorgehensweisen
tools: Read, Glob, Grep
model: sonnet
---
```

Du bist Code-Reviewer. Prüfe Code und gib konkretes, umsetzbares Feedback zu Qualität, Sicherheit und Wartbarkeit.



Workflows

Skill definiert einen Workflow mit konkreten Arbeitsschritten und den einzusetzenden Subagenten

Beispiel für eine Skill-Datei:

```
name: create-bug-workplan
description: Guarded Workflow für Bug-Analyse + Architektur-Planung.
disable-model-invocation: false
user-invocable: true
```

Starte den Guarded-Workflow für Bug-Analyse und Architektur-Planung von \$ARGUMENTS (Ticket-ID oder Ticket-Datei).

Dieser Skill läuft ausschließlich mit Subagents über das Agent-Tool.

Führe den Ablauf strikt sequentiell aus: Phase 1: Bug Analyst → Reviewer Phase 2: Architect → Reviewer

Phase 1A: Bug-Analysen Nutze den Agenten (.claude/agents/bug-analyst.md). Input: Bug-Ticket als Markdown-Datei

Aufgaben...

Phase 2A: Architektur-Planung Nutze den Agenten (.claude/agents/architect.md). Input: Bug-Analyse (bug-analysis.md), Ticket

Aufgaben...

Stop-Bedingung: Der Skill ist beendet, sobald alle folgenden Punkte erfüllt sind: Phase 1 (Bug-Analyse) ist fachlich abgeschlossen, reviewt und protokolliert. Phase 2 (Architektur-Planung) ist fachlich abgeschlossen, reviewt und protokolliert.

Eskalation: Nach 3 erfolglosen Iterationen (3× CHANGES_REQUESTED): Eskalation an Nutzer



Plugins & Plugin-Marktplätze

EINHEITLICHES PLUGIN-FORMAT

Ein Plugin bündelt MCP-Server, Skills, Subagents und Hooks in einem gemeinsamen Format. Installation und Aktualisierung erfolgen direkt per Terminalbefehl.

ENTERPRISE-MARKTPLÄTZE

Organisationen können Marktplätze aufbauen, um Plugins über Teams hinweg oder für Kund:innen bereitzustellen.

Selbst bauen & bereitstellen – oder fertige Plugins aus dem offiziellen Anthropic Plugin Marketplace und aus Community-Marktplätzen installieren.



Hands-on Teil 2





Aufgabe 3: Workflow bauen & Feature umsetzen

EUER KONKRETER AUFTRAG

Baut einen Workflow aus drei Agenten und drei Skills – und setzt damit das Ticket APP01 nachvollziehbar um.

Ergebnis: Workflow+ Applicants-Modul

1 Bausteine: 5 Dateien

- architect → plan-feature
- developer → implement-feature
- reviewer → review-feature
- feature-workflow: orchestriert

2 Ablauf bis zur Freigabe

- APP01 → Architekt: Plan + Fragen
- Mensch: ändern & freigeben
- Developer: Plan umsetzen

3 Review & Abschluss

- Reviewer liefert Findings
- Mensch wählt Findings aus
- Beheben → erneut prüfen
- Bewusst entscheiden, dann abschließen

ABNAHME

Swagger: Person anlegen → buchen → Termin zeigt Person

Danach gemeinsam den git diff lesen und erklären können, Entscheidungen zum Workflow begründen und Tests laufen lassen

OPTIONAL

review-feature erweitern
applicantId im DTO verpflichtend



GetStarted Programm für Absolvent*innen



Capgemini journey

In den ersten 4 Wochen



Capgemini starter training

In den ersten 3 Monaten



Hot topics

Während der ersten 12 Monate

- AI
- Cyber Security
- Cloud
- Customer Experience /UX
- ADMnext
- DevOps
- Agile
- Design Thinking
- Digital
- IoT



Mentoring

Während der ersten 12 Monate



Office paten

Während der ersten 12 Monate



Training curriculum

Jederzeit

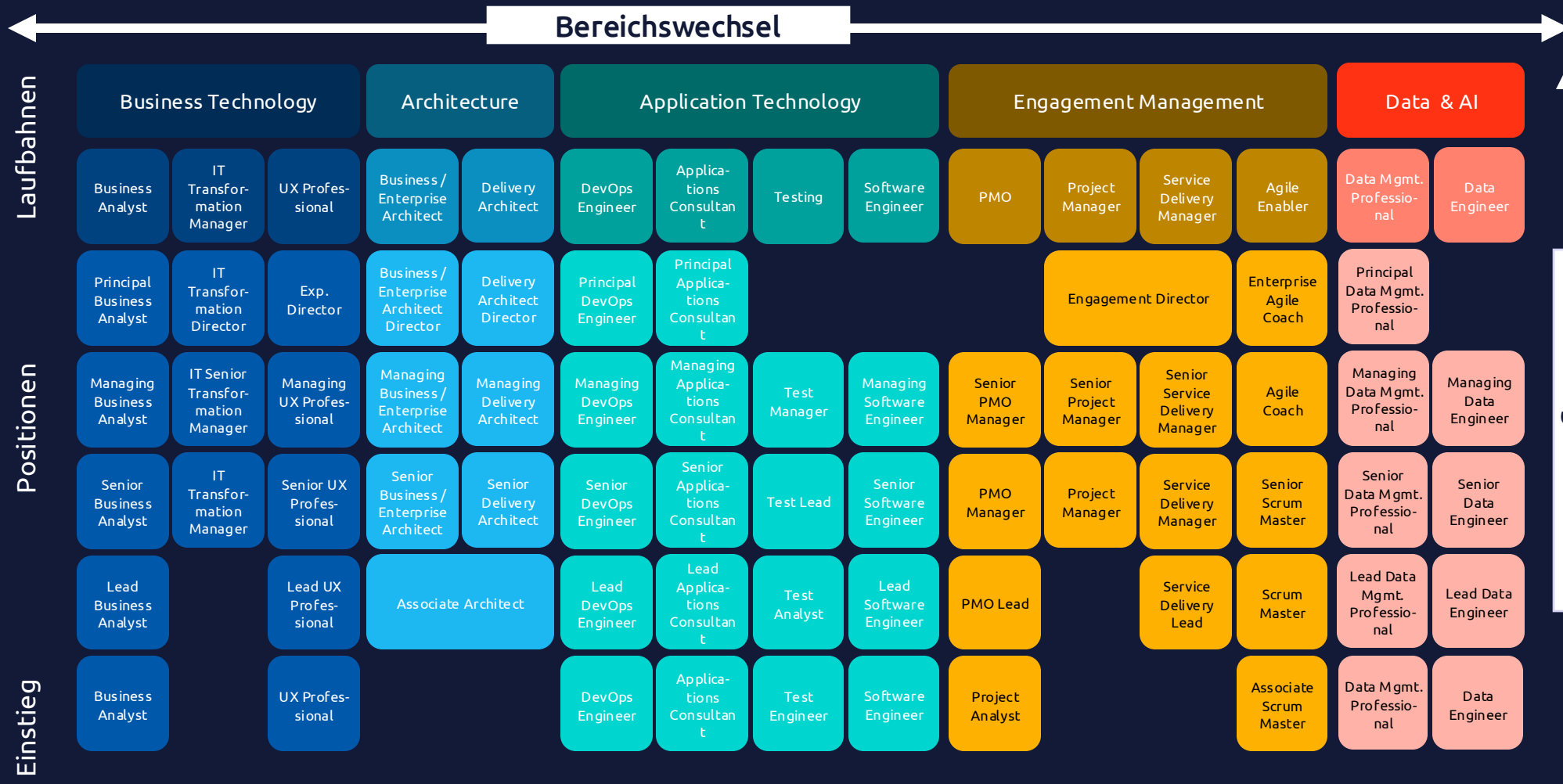
- Software Design
- JAVA
- Collaborating with clients
- ITIL
- Data Science
- Data & AI Engineering
- SAP

Deine Vorteile

- Top-Projekte
- Schwerpunkt-spezifische Trainings
- Soft Skill Trainings
- Betreuung durch Mentor*innen
- Office-Paten-Programm
- Austausch mit anderen Einsteiger*innen

Capgemini-Karrieremodell

Unser Karrieremodell und einige Möglichkeiten für deinen Laufbahnwechsel



- Karrierewege
- Aufstiegsmöglichkeiten
- Bereichswechsel
- Entwicklungsperspektiven
- Regelmäßige Trainings
- Fortbildungsmöglichkeiten

Unsere Standorte



Dein Kontakt zu uns

Capgemini Deutschland GmbH
Bahnhofstr. 30
90402 Nürnberg



career.de@capgemini.com
www.capgemini.de/karriere

Folge uns auf:



facebook.com/CapgeminiDE



instagram.com/CapgeminiDE



linkedin.com/company/Capgemini



xing.com/companies/Capgemini



<https://www.youtube.com/@CapgeminiGlobal>



GEBT UNS FEEDBACK 😊



About Capgemini

Capgemini is an AI-powered global business and technology transformation partner, delivering tangible business value. We imagine the future of organizations and make it real with AI, technology and people. With our strong heritage of nearly 60 years, we are a responsible and diverse group of over 420,000 team members in more than 50 countries. We deliver end-to-end services and solutions with our deep industry expertise and strong partner ecosystem, leveraging our capabilities across strategy, technology, design, engineering and business operations. The Group reported 2025 global revenues of €22.5 billion.

Make it real.

www.capgemini.com



This presentation contains information that may be privileged or confidential and is the property of the Capgemini Group.

Copyright © 2026 Capgemini. All rights reserved.